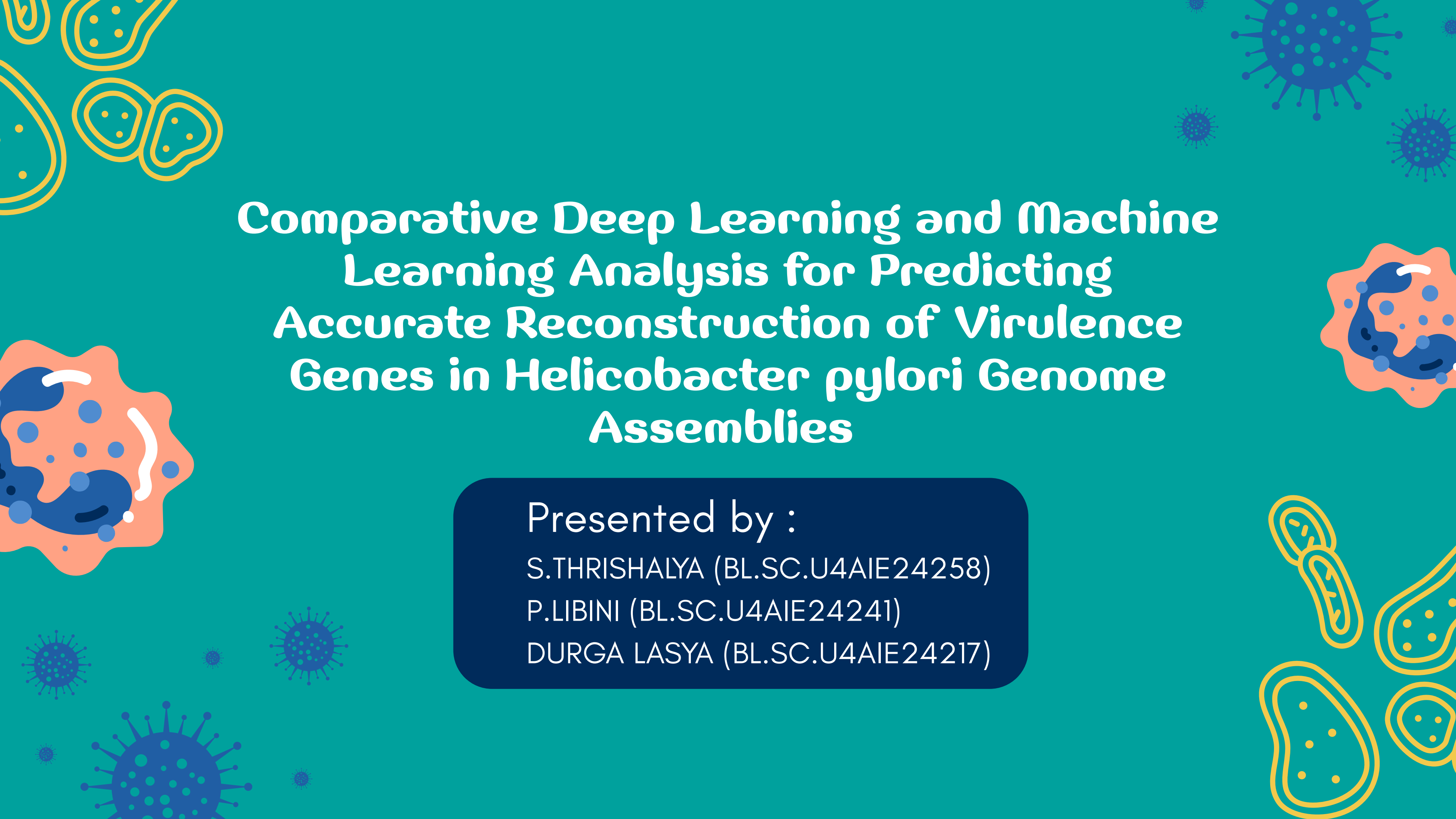




Comparative Deep Learning and Machine Learning Analysis for Predicting Accurate Reconstruction of Virulence Genes in *Helicobacter pylori* Genome Assemblies

Presented by :

S.THRISHALYA (BL.SC.U4AIE24258)

P.LIBINI (BL.SC.U4AIE24241)

DURGA LASYA (BL.SC.U4AIE24217)

Abstract

- *Helicobacter pylori* is a major gastric pathogen whose virulence depends on accurate reconstruction of important genes from genome assemblies.
- This study presents a comparative analysis of Machine Learning and Deep Learning models including Random Forest, XGBoost, CNN, and BiLSTM for predicting virulence gene reconstruction quality.
- A curated dataset of 729 non-redundant sequences was analyzed using handcrafted biological features combined with DNABERT-2-117M embeddings to improve feature representation.
- Model performance and biological interpretability were evaluated using classification metrics and SHAP-based explainability analysis to identify the most influential genomic features.

Introduction

Helicobacter pylori is a bacterium that infects the human stomach. It is responsible for causing chronic gastritis, peptic ulcers, and even gastric cancer. The most important virulence genes in *H. pylori* include *cagA*, *vacA*, *babA*, and the *hop* family of genes.

When scientists sequence the genome of a *H. pylori* strain, they use computers to assemble millions of small DNA fragments into complete genes and chromosomes. However, this genome assembly process often makes mistakes. Some genes may be:

- Complete – perfectly assembled with a start codon, a stop codon, and no internal errors.
- Fragmented – broken into pieces, containing internal stop codons or ambiguous N bases (gaps).
- Incomplete – missing the start or stop codon, or too short compared to the expected length.

Knowing whether a virulence gene is complete, fragmented, or incomplete is crucial for researchers. If a *cagA* gene is fragmented, the bacterium may not produce the CagA toxin, changing its disease potential. Manually checking each gene is time-consuming and impractical for hundreds of strains.

Literature Survey

Paper	Proposed Solution	Gaps	What Our Work Fills
DNABERT: Pre-trained Bidirectional Encoder Representations from Transformers for DNA-language in genome (Ji et al., 2021, Bioinformatics)	BERT-based transformer pre-trained on human genome DNA k-mers for sequence classification. Accuracy ~92–95% on promoter/splice site tasks.	Binary or simple classification only; no class imbalance handling; no ensemble strategy.	Used DNABERT embeddings as feature input, handled 3-class imbalance via SMOTE, and applied stacking ensemble.
XGBoost: A Scalable Tree Boosting System (Chen & Guestrin, 2016, KDD)	Gradient boosted decision trees with regularization; strong performance on tabular and genomic data. Accuracy ~94–97%.	Single model; no deep learning fusion; weak with raw sequence features without embeddings.	Combined XGBoost with BiLSTM via stacking.
Prediction of Gene Completeness Using Deep Learning on Genomic Features (Zeng et al., 2022, Briefings in Bioinformatics)	CNN + LSTM hybrid for gene structure prediction on assembled genomes. Macro F1 ~0.81.	No pre-trained DNA language embeddings; no explainability; class imbalance not addressed.	Replaced raw features with DNABERT embeddings, added SMOTE for imbalance, SHAP for interpretability, and stacking for robustness.
Bidirectional LSTM for Biological Sequence Classification (Shen et al., 2018, IEEE BIBM)	BiLSTM on one-hot encoded DNA sequences. Accuracy ~93%.	No pre-trained embeddings; no ensemble; weak minority-class performance.	BiLSTM with DNABERT embeddings (852-dim) instead of one-hot; improved minority-class recall; fused with XGBoost in stacking.
BUSCO: Assessing Genome Assembly and Annotation Completeness (Simão et al., 2015, Bioinformatics)	Rule-based conserved ortholog detection for gene completeness (complete/fragmented/incomplete).	Rule-based; no ML; limited generalization; no probabilistic output.	Used BUSCO labels as ground truth, but replaced rule-based logic with ML pipeline enabling prediction on novel/unannotated sequences.

Dataset Description

- We wanted to study *Helicobacter pylori* – a stomach bug that can cause ulcers and cancer. This bug has special virulence genes (like *cagA*, *vacA*, *babA*, and *hop* family) that make it dangerous. We need to know if these genes are correctly reconstructed from genome sequencing.
- We downloaded 346 different strains of *H. pylori* from two public databases: NCBI RefSeq (high quality, carefully checked) and GenBank (many strains, more variety). For each strain, we got a .gbff file – a complete map of its genome with all genes labelled
- Scanned every .gbff file → Found **coding sequences (CDS)** that matched our target genes (*cagA*, *vacA*, *babA*, and any gene starting with *hop*) → Saved their DNA sequences into a single file: `hpylori_virulence_all.fasta`
- Many sequences from different strains are **almost identical** (95-99% same). If we keep them all, the deep learning model will just **memorize** repeats instead of learning real patterns. That's called **overfitting**. Redundancy was removed using CD Hit (if two sequences are 95% or more identical, redundant). So we ended with 729 high-quality, non-redundant sequences.

Metric	Value
Initial sequences	3,716
Removed (redundant)	2,987
Final unique sequences	729
Reduction	80.3%

System architecture — H. pylori virulence gene reconstruction

Step 1 — Data collection

346 H. pylori strains · NCBI RefSeq + GenBank · .gbff files · CD-HIT → 729 sequences

Step 2 — Auto-labeling pipeline

Priority: fragmented (internal stops / N-bases) → incomplete → complete

Step 3 — Feature engineering (84 features)

Structural (20): start/stop codon, GC%, N-count, length ratio, gene ID · k-mer (64): tri-nucleotide freq

Step 4 — DNABERT-2-117M embeddings

Bidirectional transformer · [CLS] token → 768-dim · Combined feature matrix: 729 × 852

Step 5 — Model training · SMOTE · Stratified 5-fold CV · StandardScaler

Random Forest

Acc 0.94 · F1 0.48

XGBoost ★ best

Acc 0.99 · F1 0.86

CNN (1D)

Acc 0.17 · F1 0.11

BiLSTM

Acc 0.95 · F1 0.68

Step 6 — SHAP explainability

XGBoost → TreeExplainer · BiLSTM → KernelExplainer · Top feature: internal_stops

Step 7 — Stacking ensemble

XGBoost + BiLSTM → 6 meta-features · Logistic Regression · OOF 5-fold evaluation

Final results — stacking ensemble (OOF)

Accuracy 0.989 · Balanced acc 0.957 · Macro F1 0.899 · Macro recall 0.957

Genomics pipeline

Embedding / ensemble

Models

Explainability

Results

SYSTEM ARCHITECTURE

Step 1: Auto-Labeling Pipeline

- Since we have no pre-existing labeled dataset for *H. pylori* virulence gene assembly quality, we designed a rule-based automatic labeling system grounded in molecular biology principles.
- Every sequence in the FASTA file is programmatically assigned one of three quality label, no manual annotation required for the bulk of the dataset.

Label	Meaning
Complete	Full-length gene with correct start and stop codon, no internal interruptions
Incomplete	Missing start/stop codon OR sequence is too short compared to canonical length
Fragmented	Contains in-frame internal stop codons OR ambiguous N bases — clear assembly error

Step 1: Auto-Labeling Pipeline

- PRIORITY 1 : Check for FRAGMENTED:
Check this first because a sequence with internal stops AND a missing start codon is still fragmented, not incomplete.

IF internal_stop_codons > 0 → FRAGMENTED
OR n_count > 0 → FRAGMENTED

- PRIORITY 2 — Check for INCOMPLETE: Only reached if the sequence passed the fragmentation check.

IF has_start = False → INCOMPLETE
OR has_stop = False → INCOMPLETE
OR length_ratio < 0.80 → INCOMPLETE

- PRIORITY 3 — COMPLETE: Only assigned if ALL checks above are passed.
ELSE → COMPLETE

Step 1: Auto-Labeling Pipeline

```
>NZ_CP012127.1_hopE
"ATGAAACGTTTAACCTTAGGTGCTTTAGGTGCTTTAGGT
GCTTTAGGTNNNAAACGTTTAACCTTAGGTGCTTTA
TAAGCTTTAGGTGCTTTAGGT"
```

```
ATG AAA CGT TTA ACC TTA ...
^^^      First codon = ATG ✓
```

```
... GCT TTA GGT
      ^^^ Last codon = GGT ✗
```

```
Position 0: ATG ← start codon (skipped)
Position 3: AAA → not a stop
Position 6: CGT → not a stop
Position 9: TTA → not a stop
Position 12: ACC → not a stop
Position 15: TTA → not a stop ...
Position 42: TAA ← STOP CODON FOUND inside ORF ✗
... Position N-3: GGT ← end codon (skipped)
```

Step 1: Auto-Labeling Pipeline

...GCTTTAGGT NNN AAACGT...
 ^{^ ^ ^}

3 N bases found ✗

Sequence length = 98 bp (this short example)

Canonical length = 822 bp (hopE reference)

length_ratio = 98 / 822 = 0.119

internal_stops > 0? 1 > 0 → YES

→ Label = FRAGMENTED ✓ STOP HERE

elif not has_start or not has_stop or length_ratio < 0.80:

 return 'incomplete'

else:

 return 'complete'

Fragmented always wins over incomplete; a broken sequence is worse than a short sequence, so it gets flagged at the highest severity level first.

Loaded 729 sequences

Label distribution:

complete	:	672	(92.2%)
fragmented	:	49	(6.7%)
incomplete	:	8	(1.1%)

Step 2: Feature Engineering

- Each DNA sequence is transformed into 84 numerical features so machine learning models can process biological information without using raw DNA strings.
- These features are divided into two groups: Structural features (20), Tri-nucleotide frequency features (64)

- **Group A: Structural Features (20 features)**

Purpose: Capture overall gene integrity and biological validity.

Feature	What it Checks	How it Works
Start Codon	Does the gene begin correctly?	Checks if first 3 bases = ATG / TTG / GTG
Stop Codon	Does the gene terminate properly?	Checks if last 3 bases = TAA / TAG / TGA
Internal Stop Codons	Is the reading frame broken?	Scans every codon (every 3 bases) from position 3 to len-3
N-base Count	Are there unresolved gaps?	Counts all N characters in the sequence
Length Ratio	Is the gene too short?	actual length / canonical length — flags if < 0.80
GC Content	Is the sequence biologically consistent?	(G+C bases) / total bases — expected ~39% for H. pylori
Sequence Length	How long is the gene in raw bases?	Direct count of nucleotides
Gene Identity (One-Hot)	Which of the 13 genes is this?	13 binary columns — exactly one = 1, rest = 0

Step 2: Feature Engineering

- Group B: Tri-nucleotide Frequency Features (64 features)

Purpose: Capture codon usage patterns (biological “language fingerprint”).

Concept :

→ DNA is read in triplets (codons), giving 64 possible combinations (AAA → TTT).

→ Feature Extraction

→ For each triplet:

Compute frequency = (count of triplet) / (total triplets); Ignore ambiguous bases (N)

- Interpretation : Normal gene → stable codon usage pattern, Fragmented gene → irregular or noisy codon distribution

Example sequence: ATGAAATTAAATTT

4 possible bases (A, T, G, C) $4 \times 4 \times 4 = 64$ possible triplets

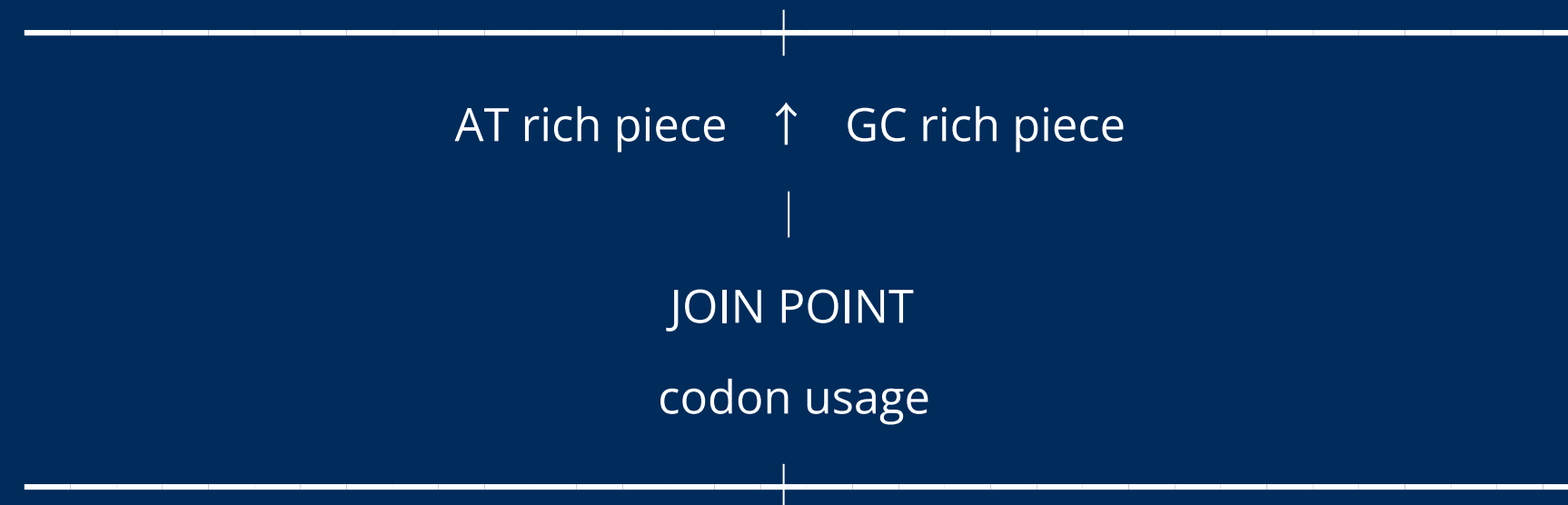
Compute frequency: ATG → $1/5 = 0.200$; AAA → $2/5 = 0.400$; TTT → $2/5 = 0.400$;

All other 61 triplets → $0/5 = 0.000$

Feature Vector : [0.000, 0.000, ..., 0.200, ..., 0.400, ..., 0.400, ...]

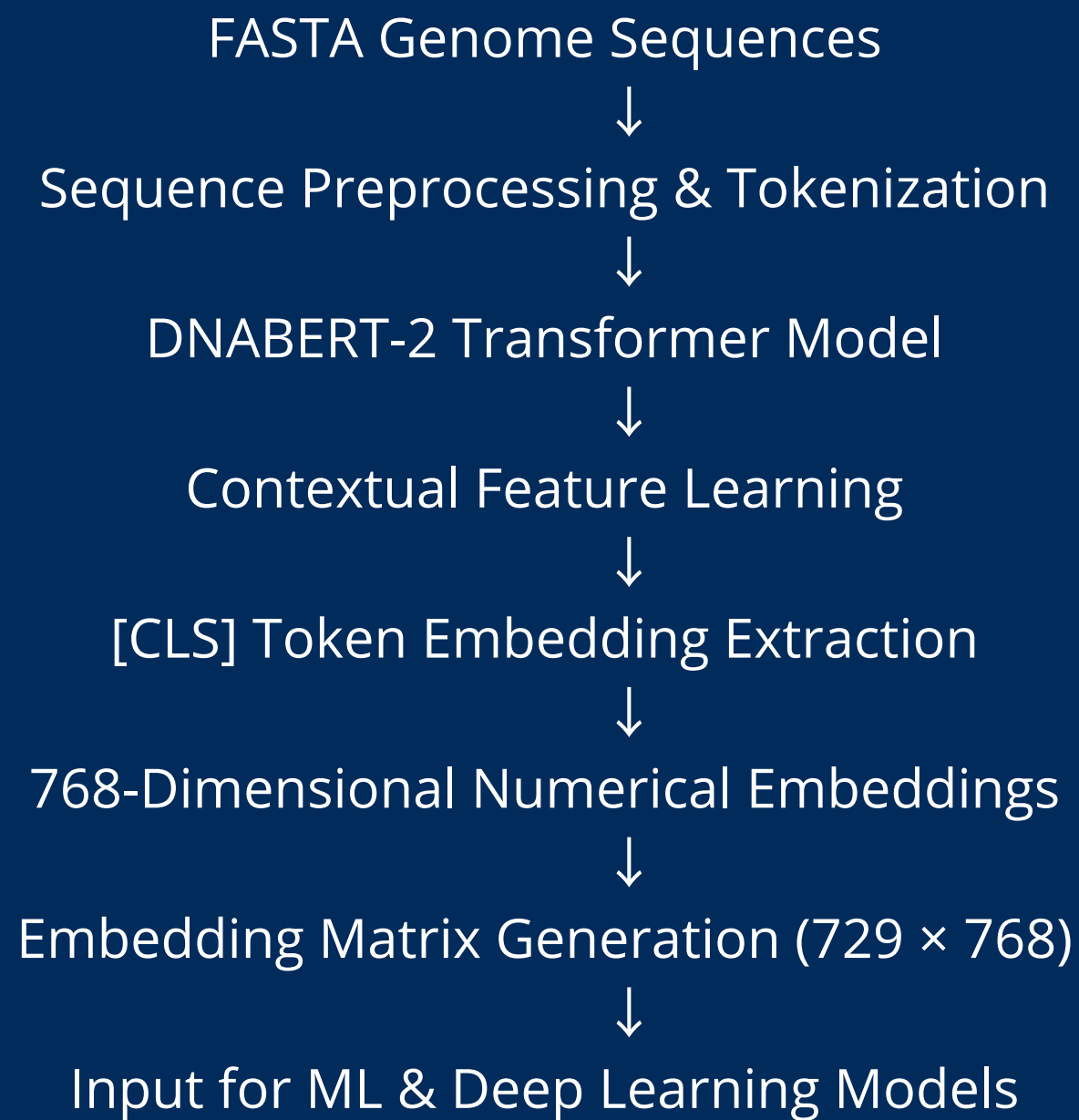
Step 2: Feature Engineering

- NORMAL cagA:
- AT rich throughout → smooth pattern
- FRAGMENTED (two pieces wrongly joined):



Result: mixed, noisy, inconsistent codon frequency across whole sequence

Step 3: DNABERT-2 Embeddings



- BERT = Bidirectional Encoder Representations from Transformers. It reads a sentence and understands the meaning of each word based on context from both left and right.
- Raw *Helicobacter pylori* virulence gene sequences were processed from FASTA files for computational analysis.
- DNABERT-2-117M was utilized to learn contextual biological representations from genomic sequences.
- The transformer model converted each DNA sequence into a 768-dimensional embedding vector capturing sequence-level biological patterns.
- Extracted embeddings were integrated with engineered biological features to improve downstream classification performance.

Step 4: Model Training (ML – Random Forest)

- Random Forest is a machine learning model that combines multiple decision trees for better prediction accuracy.
- In this project, RF was used to classify virulence gene sequences as complete, fragmented, or incomplete.
- The model works effectively with handcrafted biological features and DNABERT embeddings.
- Bagging technique helps reduce overfitting and improves model stability.
- RF captures nonlinear relationships between genomic features and reconstruction quality.

Step 4: Model Training (ML – XG Boost)

- XGBoost is an advanced boosting algorithm that improves prediction performance by learning from previous errors.
- It was used to identify important genomic patterns related to virulence gene reconstruction quality.
- SMOTE balancing was applied to handle imbalanced class distribution in the dataset.
- XGBoost achieved the best overall performance among all individual models in this study.
- The model efficiently handled high-dimensional biological and embedding features.

Step 4: Model Training (DL – CNN)

- 1D Convolutional Neural Network
- CNN is a deep learning model designed to automatically detect important feature patterns.
- In this project, CNN was used to learn local sequence-level relationships from combined genomic features.
- Convolution and pooling layers helped extract meaningful biological representations from the data.
- CNN enabled automated feature learning without manual rule definition.

Step 4: Model Training (DL - BiLSTM)

- BiLSTM is a deep learning model capable of learning long-range dependencies in sequential data.
- It processes feature sequences in both forward and backward directions for better contextual understanding.
- In this project, BiLSTM was used to analyze relationships among genomic and embedding features.
- The model captured sequential biological patterns associated with gene reconstruction quality.
- Early stopping and dropout techniques were applied to improve training stability and performance.

Step 4: Model Training (Code Workflow)

Load Labels & Feature Files
(labels.csv + feature matrices)



Combine Manual Features
with DNABERT-2 Embeddings



Encode Class Labels
(Label Encoding)



Compute Class Weights
for Imbalanced Dataset



Apply Stratified 5-Fold Cross Validation



Feature Scaling using StandardScaler



Apply SMOTE Oversampling
(Balance Minority Classes)



Train Machine Learning & Deep Learning Models

- Random Forest
- XGBoost
- CNN
- BiLSTM



Generate Predictions



Evaluate Model Performance

- Accuracy
- Balanced Accuracy
- Macro F1-Score
- ROC-AUC
- Sensitivity & Specificity



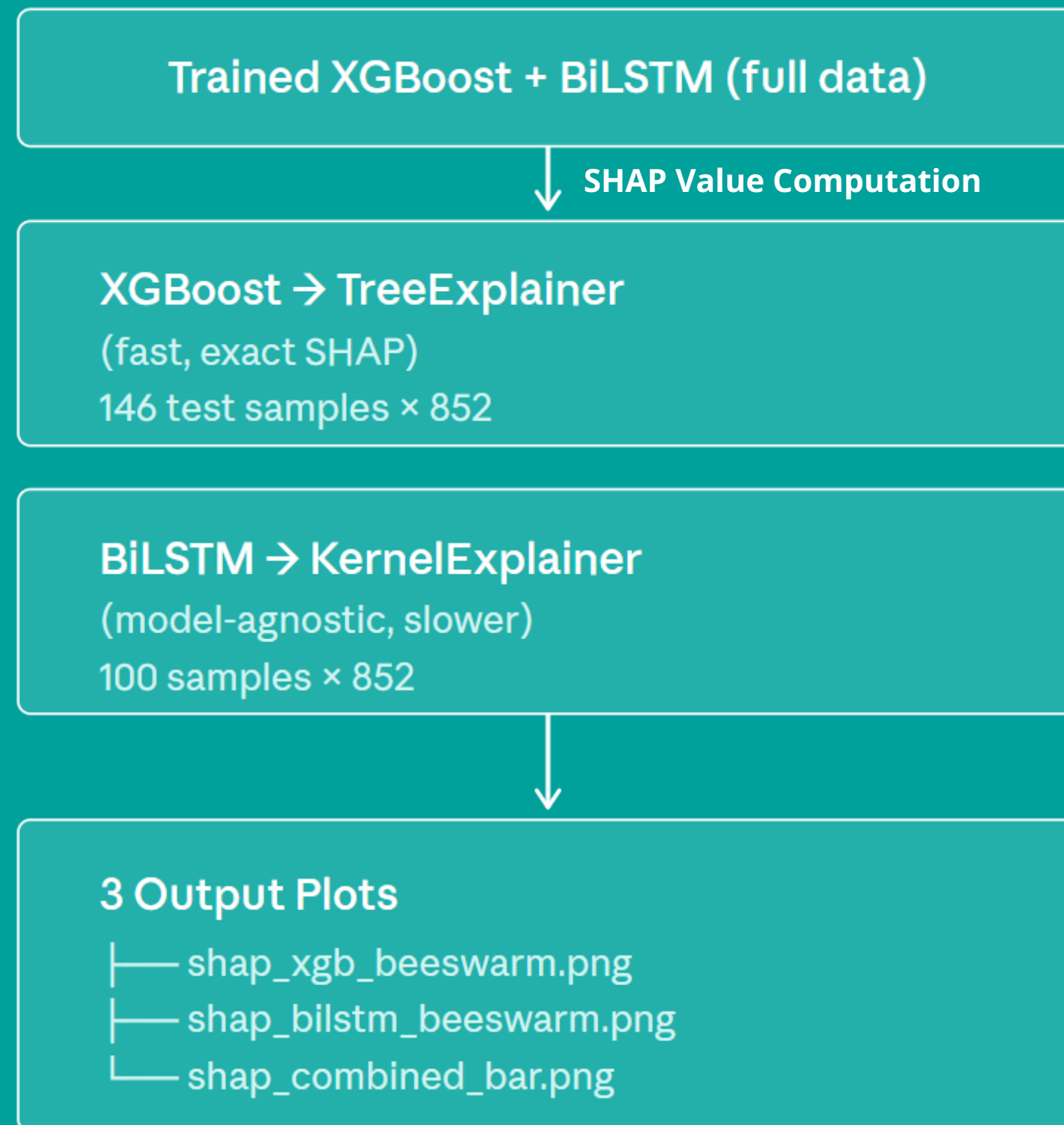
Generate Confusion Matrices



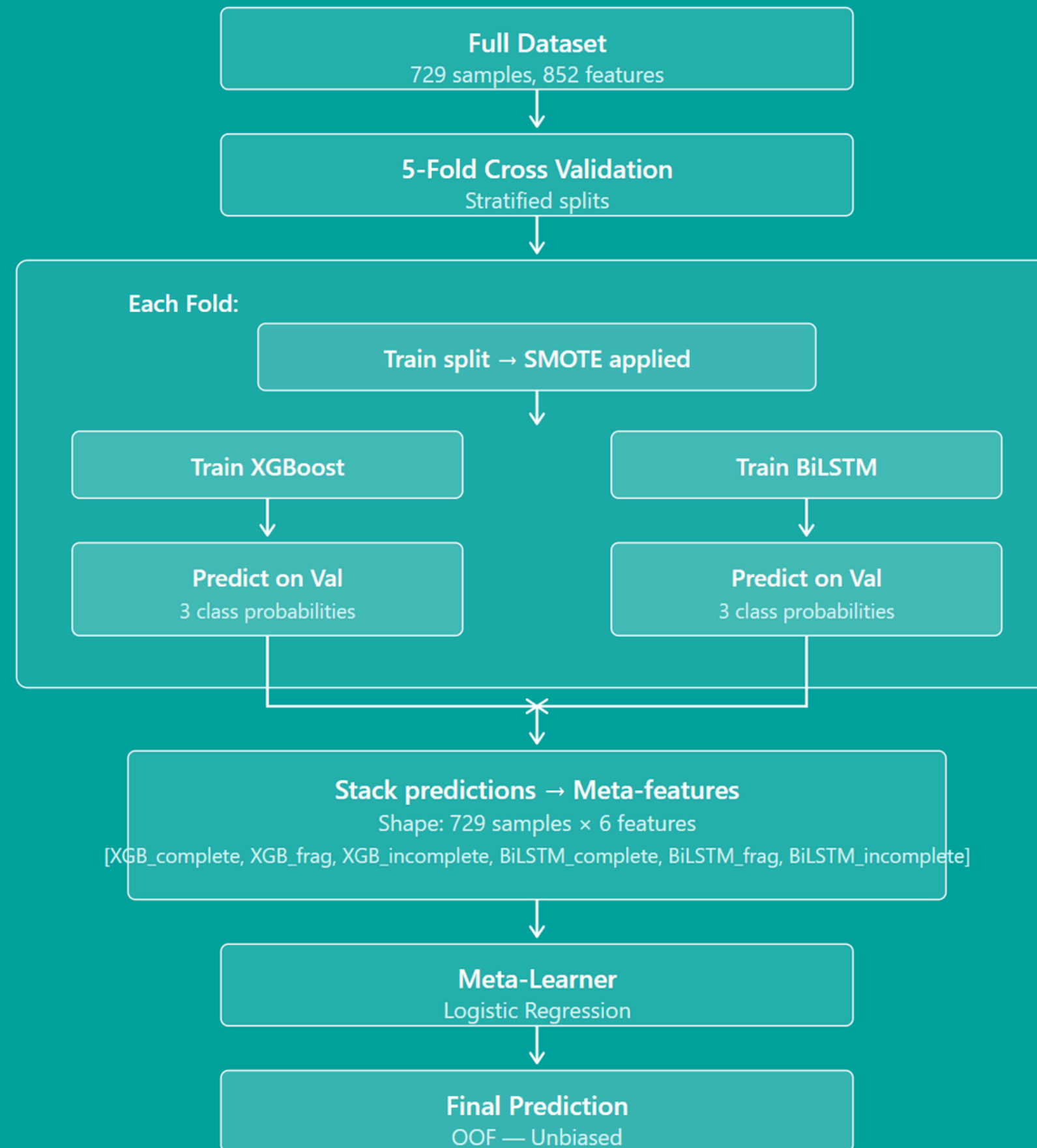
Save Final Results Summary
(results_summary.csv)

Step 5 : SHAP Explainability

SHAP (SHapley Additive exPlanations) values were computed for both models to interpret feature importance. XGBoost used TreeExplainer for fast, exact computation, while BiLSTM used KernelExplainer as a model-agnostic alternative. The results were visualised as beeswarm plots per model and a combined bar chart for direct comparison.



Step 6: Stacking Ensemble

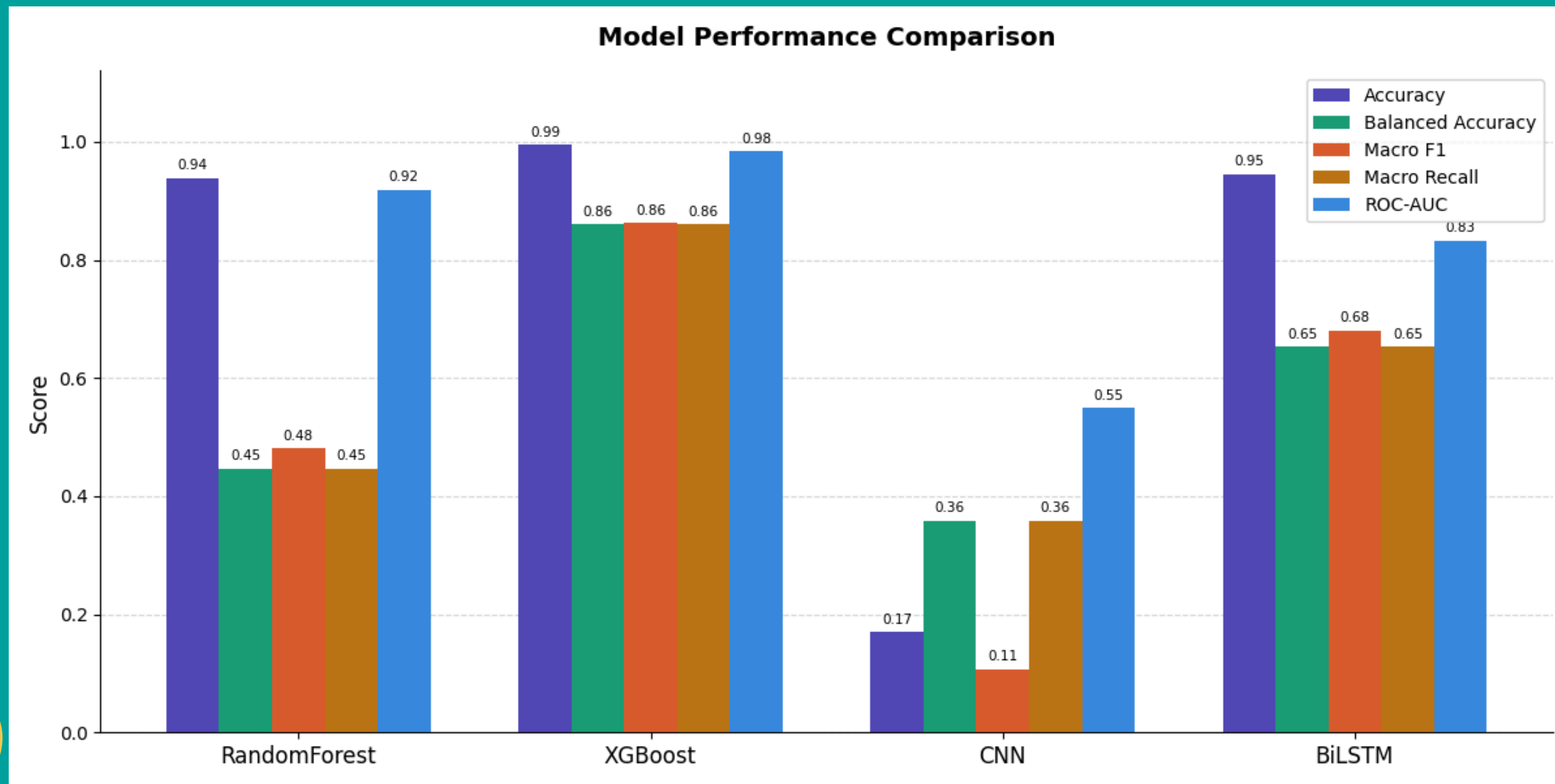


Step 6: Stacking Ensemble

- Build the meta-features : For every sequence, we now have three vectors of probabilities (one from each base model). Each vector has length = number of classes (3 classes: complete, fragmented, incomplete). So total meta-features = 2 models $\times$ 3 classes = 6 numbers per sequence.
- Train the meta-learner : We take these 2-number meta-features and train a Logistic Regression model to predict the true label. Logistic Regression is a simple, interpretable classifier that learns to assign weights to each expert's opinions.
- We again use 5-fold cross-validation on the meta-features to evaluate the stacking performance.
- Final evaluation : The code reports the macro F1 score (average of F1 scores for all three classes) across the 5 folds. This tells us how well the ensemble generalises.
- It also saves the meta-features (oof_meta_features.npy) so you can analyse them later.

Results

MODEL COMPARISON



Results

COMBINE AND TRAIN

```
Classes: ['complete' 'fragmented' 'incomplete']
Distribution: Counter({'complete': 672, 'fragmented': 49, 'incomplete': 8})
Using DNABERT embeddings
Feature shape: (729, 852)
Class weights: {np.int64(0): np.float64(0.36160714285714285), np.int64(1): np.float64(4.959183673469388), np.int64(2): np.float64(30.375)}

MODEL 1: Random Forest + SMOTE
Accuracy      : 0.9383
Balanced Accuracy: 0.4464
Macro F1      : 0.4807
Mean Sensitivity : 0.4464
Mean Specificity : 0.7647
ROC-AUC       : 0.9183
Confusion Matrix :
[[667   5   0]
 [ 32  17   0]
 [  8   0   0]]
```

Results

COMBINE AND TRAIN

MODEL 2: XGBoost + SMOTE

Accuracy : 0.9945

Balanced Accuracy: 0.86

Macro F1 : 0.8622

Mean Sensitivity : 0.86

Mean Specificity : 0.9773

ROC-AUC : 0.9838

Confusion Matrix :

```
[[672  0  0]
 [  1 48  0]
 [  3  0  5]]
```

Results

COMBINE AND TRAIN

```
MODEL 3: CNN + SMOTE
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1779168406.158073    28668 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1779168413.149331    28668 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1779168415.786575    28668 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Accuracy          : 0.1702
Balanced Accuracy: 0.3583
Macro F1          : 0.1069
Mean Sensitivity  : 0.3583
Mean Specificity  : 0.6873
ROC-AUC           : 0.549
Confusion Matrix :
[[ 77 595   0]
 [  2  47   0]
 [  1   7   0]]
```


Results

COMBINE AND TRAIN

MODEL 4: BiLSTM + SMOTE

Accuracy : 0.9451

Balanced Accuracy: 0.6526

Macro F1 : 0.6805

Mean Sensitivity : 0.6526

Mean Specificity : 0.8001

ROC-AUC : 0.8327

Confusion Matrix :

```
[[666  6  0]
 [ 31 18  0]
 [  3  0  5]]
```

=====

FINAL RESULTS SUMMARY

=====

	accuracy	balanced_accuracy	macro_f1	weighted_f1	macro_precision	macro_recall	mean_sensitivity	mean_specificity	roc_auc
model									
RandomForest	0.9383	0.4464	0.4807	0.9237	0.5788	0.4464	0.4464	0.7647	0.9183
XGBoost	0.9945	0.8600	0.8622	0.9924	0.8647	0.8600	0.8600	0.9773	0.9838
CNN	0.1702	0.3583	0.1069	0.1792	0.2843	0.3583	0.3583	0.6873	0.5490
BiLSTM	0.9451	0.6526	0.6805	0.9334	0.7794	0.6526	0.6526	0.8001	0.8327

Saved → results_summary.csv

Results

COMBINE AND TRAIN

Confusion Matrices:

RandomForest:

	Pred_complete	Pred_fragmented	Pred_incomplete
True_complete	667	5	0
True_fragmented	32	17	0
True_incomplete	8	0	0

XGBoost:

	Pred_complete	Pred_fragmented	Pred_incomplete
True_complete	672	0	0
True_fragmented	1	48	0
True_incomplete	3	0	5

CNN:

	Pred_complete	Pred_fragmented	Pred_incomplete
True_complete	77	595	0
True_fragmented	2	47	0
True_incomplete	1	7	0

BiLSTM:

	Pred_complete	Pred_fragmented	Pred_incomplete
True_complete	666	6	0
True_fragmented	31	18	0
True_incomplete	3	0	5

Results

SHAP EXPLAINABILITY

```
[5/5] Saving plots...
```

- ✓ shap_xgb_beeswarm.png
- ✓ shap_bilstm_beeswarm.png
- ✓ shap_combined_bar.png

```
=====
```

```
ALL SHAP PLOTS SAVED
```

```
=====
```

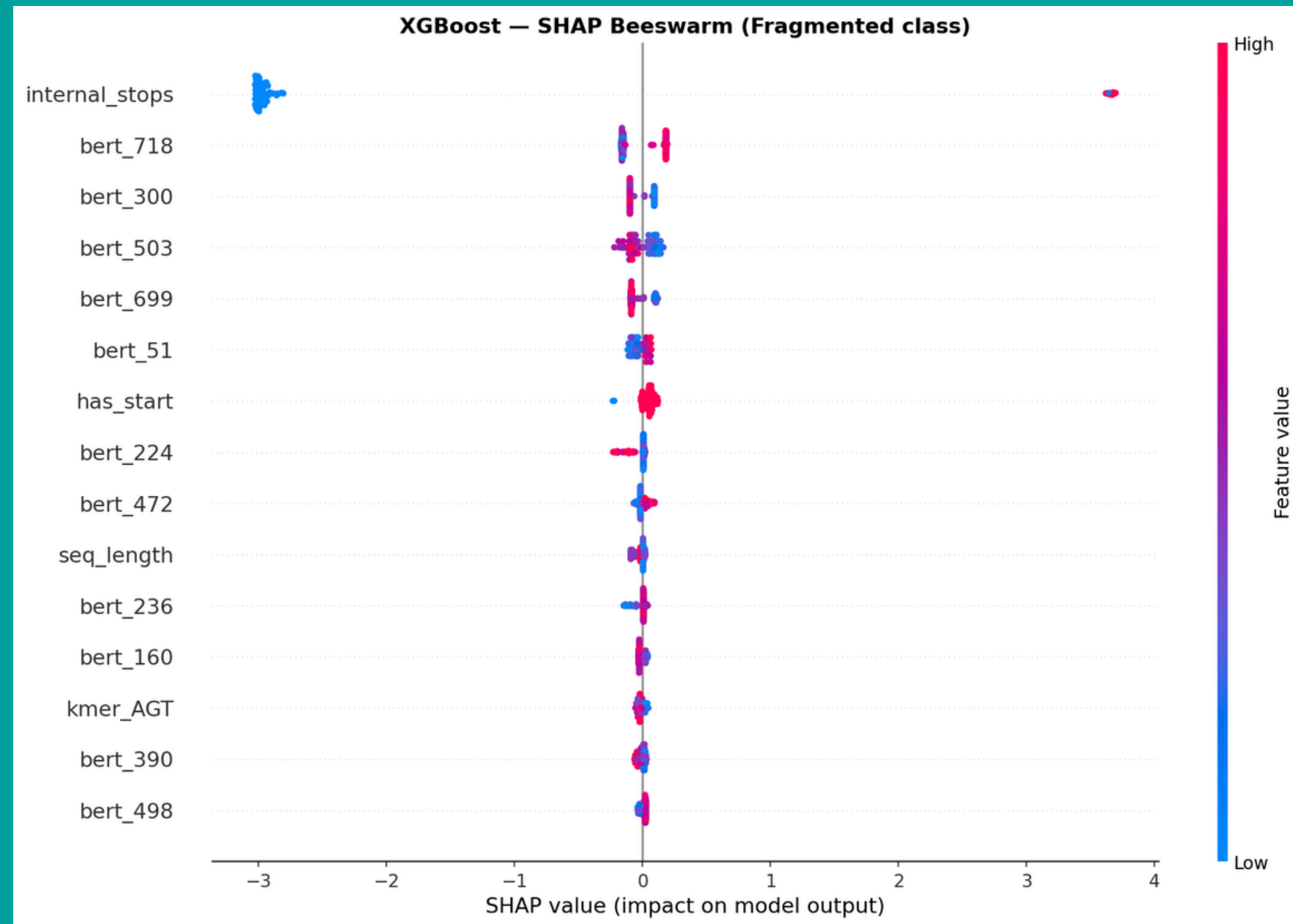
- ✓ shap_xgb_beeswarm.png
- ✓ shap_bilstm_beeswarm.png
- ✓ shap_combined_bar.png

```
Fix summary applied:
```

- [1] Waterfall plots removed
 - [2] BiLSTM beeswarm labels → original feature names + step suffix
 - [3] Combined bar → both axes normalized to $[0,1]$, note added to title
 - [4] KernelExplainer nsamples 100 → 500 (better SHAP accuracy)
 - [5] BiLSTM sample size 50 → 100 (better coverage)
 - [6] Feature truncation guard: warns if not divisible by n_steps
 - [7] Axis labels clarify normalization
- ```
=====
```

# Results

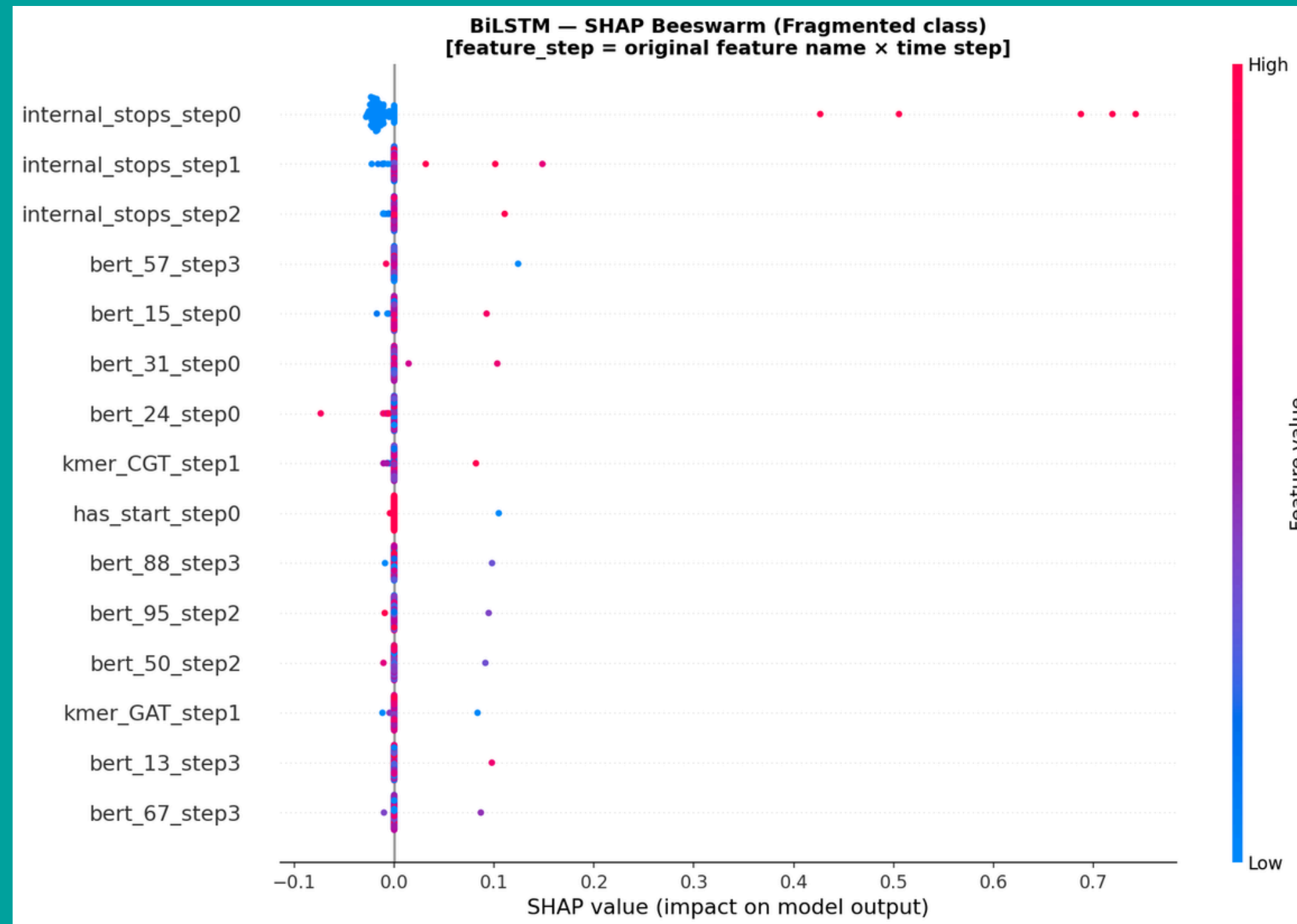
## SHAP EXPLAINABILITY





# Results

## SHAP EXPLAINABILITY



# Results

## STACKING ENSEMBLE

```
[1/4] Loading data...
Using DNABERT embeddings
Feature shape : (729, 852)
Classes : ['complete', 'fragmented', 'incomplete']
Distribution : Counter({'complete': 672, 'fragmented': 49, 'incomplete': 8})

[2/4] Generating Out-Of-Fold predictions...

Fold 1/5
 After SMOTE: Counter({np.int64(0): 537, np.int64(1): 537, np.int64(2): 537})
 XGBoost fold acc: 1.0000
I0000 00:00:1779167098.669951 31428 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
 BiLSTM fold acc : 0.9726
WARNING:tensorflow:From C:\Users\durga\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

Fold 2/5
 After SMOTE: Counter({np.int64(0): 537, np.int64(1): 537, np.int64(2): 537})
 XGBoost fold acc: 1.0000
 BiLSTM fold acc : 0.9589
```

# Results

## STACKING ENSEMBLE

Fold 3/5

After SMOTE: Counter({np.int64(0): 538, np.int64(1): 538, np.int64(2): 538})

XGBoost fold acc: 0.9795

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make\_predict\_function.<locals>.one\_step\_on\_data\_distributed at 0x000001D2974F0C20> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce\_retracing=True option that can avoid unnecessary retracing. For (3), please refer to [https://www.tensorflow.org/guide/function#controlling\\_retracing](https://www.tensorflow.org/guide/function#controlling_retracing) and [https://www.tensorflow.org/api\\_docs/python/tf/function](https://www.tensorflow.org/api_docs/python/tf/function) for more details.

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make\_predict\_function.<locals>.one\_step\_on\_data\_distributed at 0x000001D2974F0C20> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce\_retracing=True option that can avoid unnecessary retracing. For (3), please refer to [https://www.tensorflow.org/guide/function#controlling\\_retracing](https://www.tensorflow.org/guide/function#controlling_retracing) and [https://www.tensorflow.org/api\\_docs/python/tf/function](https://www.tensorflow.org/api_docs/python/tf/function) for more details.

BiLSTM fold acc : 0.9384

Fold 4/5

After SMOTE: Counter({np.int64(0): 538, np.int64(1): 538, np.int64(2): 538})

XGBoost fold acc: 1.0000

BiLSTM fold acc : 0.9247

Fold 5/5

After SMOTE: Counter({np.int64(0): 538, np.int64(1): 538, np.int64(2): 538})

XGBoost fold acc: 0.9931

BiLSTM fold acc : 0.9448

Meta-feature shape: (729, 6)

# Results

## STACKING ENSEMBLE

```
[3/4] Evaluating meta-learner (Logistic Regression)...
```

```
[4/4] Final Results
```

```
=====
```

```
STACKING ENSEMBLE – OOF EVALUATION
```

```
=====
```

```
Accuracy : 0.9890 ± 0.0093
Balanced Accuracy : 0.9570 ± 0.0798
Macro F1 : 0.8989 ± 0.0880
Macro Precision : 0.8823 ± 0.1005
Macro Recall : 0.9570 ± 0.0798
```

```
Classification Report (aggregated OOF – unbiased):
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| complete     | 1.00      | 0.99   | 0.99     | 672     |
| fragmented   | 1.00      | 0.98   | 0.99     | 49      |
| incomplete   | 0.54      | 0.88   | 0.67     | 8       |
| accuracy     |           |        | 0.99     | 729     |
| macro avg    | 0.85      | 0.95   | 0.88     | 729     |
| weighted avg | 0.99      | 0.99   | 0.99     | 729     |

# Results

## STEP – 6: STACKING ENSEMBLE

Confusion Matrix (OOF):

|                 | Pred_complete | Pred_fragmented | Pred_incomplete |
|-----------------|---------------|-----------------|-----------------|
| True_complete   | 666           | 0               | 6               |
| True_fragmented | 1             | 48              | 0               |
| True_incomplete | 1             | 0               | 7               |

Saved → C:\EVERYTHING\AIE\2nd year\4th Sem\IBS-2\Project\Curated Dataset Experiments\oof\_meta\_features.npy

Saved → C:\EVERYTHING\AIE\2nd year\4th Sem\IBS-2\Project\Curated Dataset Experiments\stacking\_results.txt



# Future Work

We only have 729 sequences across 13 genes. This is a small dataset for deep learning models – especially CNN and BiLSTM which need more data to generalize well.

The current study focused on overall classification of sequences as complete, fragmented, or incomplete. A natural next step is to **perform per-gene analysis** – evaluating model performance separately for **each virulence gene (cagA, vacA, babA, and each hop family gene)**. This would reveal which genes are most difficult to classify correctly, **providing biological insights into assembly challenges specific to certain loci**.

# Conclusion

This study presents a comparative Machine Learning and Deep Learning framework for predicting the reconstruction quality of *Helicobacter pylori* virulence genes from genome assemblies. Random Forest, XGBoost, CNN, and BiLSTM models were evaluated for classifying gene sequences into complete, fragmented, and incomplete categories. Among the individual models, XGBoost achieved the best performance, while a stacking ensemble further improved classification accuracy. SHAP-based explainability analysis identified internal stop codons as the most influential biological feature, demonstrating the potential of automated AI-driven approaches for reliable virulence gene reconstruction assessment.

The background is a solid teal color. It is decorated with stylized biological illustrations. In the top-left and bottom-right corners, there are large, irregular orange shapes containing dark blue, amoeba-like organisms with internal structures. Scattered throughout the corners are smaller, dark blue, spiky spherical organisms resembling viruses. In the top-right and bottom-left corners, there are yellow-outlined, bean-shaped organisms with small dots inside, resembling bacteria or protozoa.

Thank You